Which Predictions Are Worth Updating? Budgeted Predictive Knowledge for Long-Horizon Agents

Ayush Bodade
Independent Researcher
ayush@latentpaths.com
Project website

Abstract

Long-horizon agents rely on predictions about their environment, but these predictions can become stale as the environment changes. When fresh evidence is limited, which predictions should an agent update? We study this question through budgeted predictive-knowledge updates: selecting a fixed number of stored predictions to revise before a fixed LLM policy continues acting. We develop three selection strategies that prioritize predictions by change risk and downstream importance, adapt priorities using execution feedback and unfinished work, or select updates jointly to account for shared prerequisites. Across 1,872 episodes in two controlled task families and four LLM policies, combining change risk with downstream importance improves the fraction of jobs completed over random selection by 7.6–10.9 percentage points on the primary model. A follow-up study shows that accounting for remaining dependencies improves progress under execution feedback, without establishing an additional benefit from joint selection. These findings show how dependency information can guide limited knowledge updates, while revealing a persistent gap between better intermediate progress and reliable task completion.

1 Introduction

Long-horizon agents use predictions about their environment to select actions. As the environment changes, predictions can become invalid or irrelevant to unfinished goals. With limited fresh evidence, which predictions should be updated? We store each prediction in a *record*; a *selector* chooses which records receive evidence for a knowledge update. We use two workflows: DAG (a directed acyclic graph of token requirements) and Precedence Chains (jobs must also follow chain order).

Stateful Knowledge Learning (SKL) learns state-grounded predictive assessments from experience (Song et al., 2026). Its update schedules motivate our allocation question, but our experiments update component records with fixed LLM policies; they are not a reproduction of SKL training (Appendix L).

We study three classes of update-selection strategies. **Static priorities** combine change risk and downstream importance. **Feedback-conditioned priorities** incorporate execution outcomes and unfinished work. **Joint selection** accounts for predictions that must be updated together.

We formulate budgeted predictive-knowledge updates and compare these three classes within a shared policy–updater interface. We provide two controlled task families and 1,872 episodes across four LLM policies to distinguish progress, episode success, and inference cost. Code and data reproduce the reported results without new model calls.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Neuro-Symbolic Embodied Intelligence.

2 Problem Formulation

A fixed LLM policy chooses actions using its interaction history and predictive knowledge K, represented by n stored predictions. At an update step, a selector chooses exactly k records, forming the set U. Only those records receive fresh evidence, which an LLM updater uses to revise K before the policy continues. A trajectory is the sequence of actions and observations (including verifier feedback) within one episode. No model weights change. The budget counts records, not inference tokens.

3 Update-Selection Strategies

Individual priorities. The first two families rank records independently and select the top k:

$$S(i) = r_i w_i, \qquad r_i : \text{ staleness factor}, \quad w_i : \text{ downstream importance}. \tag{1}$$

Setting either factor to one ignores that signal. Table 1 shows the five choices; Joint instead scores a complete set of updates. Appendix M walks through all three families numerically.

Table 1: Update-selection strategies. Blue marks methods studied as part of our framework. R: risk; D: dependency; F: execution feedback. The same names label results.

Family	Selector	r_i	w_i
Static	Risk (R)	p_i	1
Static	Dependency (D)	1	$C(i)$
Static	Risk $\times$ Dependency (RD)	p_i	$C(i)$
Feedback	Feedback-Risk (FR)	$1 - q_i$	1
Feedback	Feedback-Risk $\times$ Dependency (FRD)	$1 - q_i$	$D_t(i)$
Joint	Joint	maximize $F(U)$ (Eq. 2)	

Static priorities. Let p_i be the supplied probability that record i changes at an environment state-change event. An edge $i \to j$ means subgoal j depends on state i. We measure static importance by $C(i) = \sum_{u \in \{i\} \cup \mathrm{Desc}(i)} 0.9^{d(i,u)}$, counting each descendant once at shortest distance $d(i, u)$. R targets likely changes, D targets downstream use, and RD combines both. RD approximates an additive loss from stale records, analogous to gain–need prioritization (Dayan, 1993; Mattar and Daw, 2018); it uses supplied structure, not a learned representation.

Feedback-conditioned priorities. Let $q_i \in [0, 1]$ estimate whether stored record i is valid, $\mathcal{P}$ be unfinished subgoals, and A_j the records required by subgoal j. Then $D_t(i) = |\{j \in \mathcal{P} : i \in A_j\}|$ counts remaining demand. FR targets estimated staleness; FRD weights it by that demand. In DAG, A_j contains the subgoal's own state and direct dependencies. In Precedence Chains it also includes requirements of unfinished predecessors. Initially $q_i = 1 - p_i$; a new state-change event multiplies it by $1 - p_i$. Acceptance or rejection of a submitted stored value sets it to one or zero. Feedback reveals errors, not replacement values; only selected records receive fresh targets. Updater errors remain in the stored predictions (Appendix K). These estimates are not claimed calibrated posteriors.

Joint selection. If a restart needs two stale records, updating either alone may not help. Joint scores the expected number of unfinished subgoals with valid requirements, assuming independent record validity and successful selected updates:

$$F(U) = \sum_{j \in \mathcal{P}} \prod_{i \in A_j \setminus U} q_i, \qquad U^* \in \arg\max_{|U|=k} F(U). \tag{2}$$

For $A_j = \{a, b\}$ and $q_a = q_b = 0$, either update alone contributes zero, but both contribute one. More generally, F is monotone and supermodular: an update's marginal value can increase as others are selected (proof in Appendix K). We enumerate $\binom{24}{4} = 10{,}626$ sets with seeded ties. This is a requirement-validity proxy, not a guarantee of completed actions; it omits action limits and model errors.

4 Experiments

Task and interaction. Each episode is a synthetic versioned workflow with 24 jobs (the subgoals) and 24 hidden access tokens. Token i has the form `mode@revision`; its record stores the agent's current prediction. The policy sees JSON listing jobs, token requirements, stored predictions, completed jobs, remaining actions, and recent feedback, but not hidden values. A job action submits certificates for its own token and every directly required token. The verifier accepts only if all certificates match the hidden current values.

Public	job 4 requires states $[4, 1, 3]$; stored tokens are `4:blue@4`, `1:amber@2`, `3:green@1`.
Action	submit job 4 with those three tokens as certificates.
Hidden	a state-change event has changed state 1 to `violet@3`.
Feedback	reject job 4; report stale state $[1]$, but withhold its current token.
Update	with $k = 1$, the selector may choose state 1; only then is fresh evidence supplied to the updater.

A state change replaces the mode and increments the revision, making the stored token stale.

Two dependency structures. In **DAG**, each job requires its own token and up to two earlier tokens, but jobs have no completion order. In **Precedence Chains**, four independent six-job chains require both the predecessor's token and prior completion; for example, chain $0 \to 4 \to 8 \to 12 \to 16 \to 20$ blocks its remaining suffix when an early job fails. The name denotes this ordering constraint; the benchmark does not simulate physical repair.

Knowledge-update budget and outcomes. The policy has 32 environment steps in eight batches of at most four actions; incorrect or malformed actions consume steps. State-change events occur before batches one and five, each followed by an update step. Component change probabilities are drawn from $\mathcal{U}(0.02, 0.30)$ independently of graph position and supplied to selectors. *Progress* is the fraction of 24 jobs completed; *episode success* requires all 24, and completed jobs remain complete. Token counts include policy and updater calls. Within each model, selectors share the policy, updater, graph, and evidence format; feedback methods apply the same validity rule along their own trajectories.

Experiment	Policies	k	Episodes
Budget allocation	Qwen3-30B	2, 4, 7	816
Missing dependencies	Qwen3-30B	4	96
Cross-model comparison	GPT-4.1-mini, GLM-5.3-Flash, Qwen3.6	4	720
Feedback and joint selection	GPT-4.1-mini, Qwen3.6	4	240

k: budgeted record updates; controls use 0 or 24 records. Reused Qwen3-30B references are counted once. Selectors share paired seeds within each model and task. Feedback uses 12 fresh seeds per task and model, with protocol fixed before outcomes. All outcomes are retained; intervals use 10,000 paired-seed bootstrap resamples without multiplicity adjustment.

5 Results

Table 2: Budget allocation: Qwen3-30B progress (%), 24 paired seeds per task, $k = 4$ per update step. Blue marks our selectors; bold marks the largest budgeted mean. Random and Even are unprioritized controls; All updates 24 records.

Task	Random	Even	R	D	RD	All
DAG	46.0	39.8	51.7	47.0	**56.9**	98.4
Precedence Chains	54.7	50.0	54.9	54.9	**62.3**	98.6

Budget allocation. RD has the highest budgeted mean (Table 2), exceeding Random by 10.9 points [+1.9, +20.3] on DAG and 7.6 points [+0.2, +15.1] on Precedence Chains (95% CI). Successes remain 0/24 and 1/24, equal to Random (all budgets: Appendix C).

Missing dependencies. Mean RD progress falls at both deletion levels in both tasks; D changes in both directions (paired subset: Appendix D).

Cross-model comparison. All six RD–Random intervals for GPT-4.1-mini, GLM-5.3-Flash, and Qwen3.6-35B-A3B include zero (Appendix E). Gains across policies remain unresolved.

Table 3: Feedback and joint selection: progress (%) with successes out of 12 in parentheses, $k = 4$. RD is static; — is untested. Blue marks our selectors; bold marks the best mean or a difference whose paired 95% interval excludes zero. Δ is Joint minus FR, in percentage points.

Task	RD	FR	FRD	Joint	Δ vs. FR ↑
GPT-4.1-mini					
DAG	66.0 (0)	69.4 (1)	81.6 (2)	**84.0** (0)	**+14.6 [+5.2, +24.7]**
Precedence Chains	49.3 (0)	66.3 (0)	80.9 (0)	**81.9** (0)	**+15.6 [+7.3, +23.6]**
Qwen3.6-35B					
DAG	50.3 (0)	63.9 (0)	—	**80.6** (1)	**+16.7 [+8.3, +24.7]**
Precedence Chains	44.1 (0)	60.8 (0)	—	**75.0** (0)	**+14.2 [+4.2, +24.0]**

Feedback and joint selection. GPT-4.1-mini's FRD exceeds FR by 12.2 [+4.5, +20.1] points on DAG and 14.6 [+6.9, +21.5] on Precedence Chains (95% CI). Weighting remaining demand improves progress under the same validity-update rule. Joint also gains 14.6 and 15.6 points for GPT-4.1-mini, and 16.7 and 14.2 points for Qwen3.6 over FR (Table 3); all four intervals exclude zero. Appendix F reports all conditions. Joint exceeds static RD by 18.1–32.6 points, but that comparison changes both feedback use and selection objective.

Joint versus individual selection. Joint adds 2.4 points (95% CI [-1.7,+7.3]) on DAG and 1.0 points (95% CI [-2.4,+4.5]) on Precedence Chains over FRD. Both intervals include zero; FRD was tested only on GPT-4.1-mini. Joint's additional benefit remains unresolved.

Episode success and inference cost. Joint finishes 1/48 episodes, FR 1/48, and all-record updating 48/48; GPT's FRD finishes 2/24. Joint takes 1.7–2.6 ms per update step locally, but uses more inference tokens than full updating in all four conditions.

6 Related Work

State-grounded prediction underlies predictive knowledge (Sutton, 1988; Sutton et al., 2011). Predictive state and successor representations encode future observations or occupancy (Littman et al., 2001; Dayan, 1993). Prioritized sweeping and replay allocate backups (Moore and Atkeson, 1993; Schaul et al., 2016; Mattar and Daw, 2018); metareasoning and information gathering value computations or observations (Russell and Wefald, 1991; Hay et al., 2012; Krause and Guestrin, 2005; Golovin and Krause, 2011). Nonadditive information value is classical (Howard, 1966). Temporal abstraction and delayed-credit methods address other decisions (Sutton et al., 1999; Arjona-Medina et al., 2019). Our framework applies these allocation ideas to explicit LLM-consumed records: static and feedback families rank individual updates, while Joint represents their interactions. Our contribution is this formulation and its controlled evaluation, separating the value of dependency information from any additional benefit of selecting complementary updates together.

7 Discussion and Conclusion

The synthetic tasks supply risks, graphs, and validity feedback. They do not establish physical embodiment, learned dependencies, calibrated beliefs, or improvements to SKL training. The follow-up has only 12 seeds per condition; Joint scales combinatorially. The findings concern allocation under a record budget, with all measured policy and updater errors retained.

The experiments establish progress gains on the primary policy and a benefit from remaining dependency demand under feedback. Gains across policies and an additional benefit from Joint remain unresolved. Learned signals and reliable episode completion need further evaluation.

Acknowledgments

The author thanks Prime Intellect for providing inference credits used in this study.

References

Richard S. Sutton. Learning to Predict by the Methods of Temporal Differences. Machine Learning, 3(1):9–44, 1988.

Richard S. Sutton, Joseph Modayil, Michael Delp, Thomas Degris, Patrick M. Pilarski, Adam White, and Doina Precup. Horde: A Scalable Real-time Architecture for Learning Knowledge from Unsupervised Sensorimotor Interaction. AAMAS, pages 761–768, 2011.

Michael L. Littman, Richard S. Sutton, and Satinder P. Singh. Predictive Representations of State. NeurIPS, pages 1555–1561, 2001.

Peter Dayan. Improving Generalization for Temporal Difference Learning: The Successor Representation. Neural Computation, 5(4):613–624, 1993.

Stuart Russell and Eric Wefald. Principles of Metareasoning. Artificial Intelligence, 49(1–3):361–395, 1991.

Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting Computations: Theory and Applications. UAI, pages 346–355, 2012.

Andreas Krause and Carlos Guestrin. Near-optimal Nonmyopic Value of Information in Graphical Models. UAI, pages 324–331, 2005.

Ronald A. Howard. Information Value Theory. IEEE Transactions on Systems Science and Cybernetics, 2(1):22–26, 1966.

Daniel Golovin and Andreas Krause. Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization. JAIR, 42:427–486, 2011.

Andrew W. Moore and Christopher G. Atkeson. Prioritized Sweeping: Reinforcement Learning With Less Data and Less Time. Machine Learning, 13(1):103–130, 1993.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized Experience Replay. ICLR, 2016.

Marcelo G. Mattar and Nathaniel D. Daw. Prioritized Memory Access Explains Planning and Hippocampal Replay. Nature Neuroscience, 21:1609–1617, 2018.

Yan Song, Xidong Feng, Bo Liu, Xinyu Cui, Haotian Fu, Zichen Liu, Mengyue Yang, Cheng Deng, Jian Zhao, and Jun Wang. Learning Stateful Predictive Knowledge From Experience. ICML AIWILD Workshop, 2026. arXiv:2607.28638.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. Artificial Intelligence, 112(1–2):181–211, 1999.

Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. RUDDER: Return Decomposition for Delayed Rewards. NeurIPS, 2019.

A Predictive-knowledge update algorithm

Algorithm 1 Predictive-knowledge updates with static or feedback-conditioned selection

Require: predictive knowledge K, risks p, graph G, domain, budget k, selector m, updater $\mathcal{M}$
Ensure: updated predictive knowledge K and executed trajectory

Apply the first exogenous change; initialize $q_i \leftarrow 1 - p_i$
for $b \in \{1, 2\}$ **do**
  if $b = 2$ **then**
    Apply the next exogenous change; $q_i \leftarrow q_i(1 - p_i)$
  end if
  $\mathcal{P} \leftarrow$ unfinished subgoals
  For $j \in \mathcal{P}$, set A_j to j's own state and direct parents
  if domain is Precedence Chains **then**
    Recursively add requirements of unfinished predecessors
  end if
  Select U with $|U| = k$ (seeded ties; Joint tolerance 10^{-12})
  static family: R, D, RD rank p_i, $C(i)$, $p_i C(i)$; random/even controls
  feedback family: FR ranks $1 - q_i$; FRD ranks $(1 - q_i)D_t(i)$
  joint family: enumerate size-k sets maximizing $F(U)$
  Obtain validated targets E_U only for selected records
  $\widehat{K}_U \leftarrow \mathcal{M}(E_U, K_U)$
  for $i \in U$ **do**
    If $\widehat{K}_i$ is a parsed string, $K_i \leftarrow \widehat{K}_i$; else retain K_i
    $q_i \leftarrow 1$ if K_i matches the validated target, else 0
  end for
  for four batches of at most four actions, with feedback after each batch **do**
    Execute actions and record newly completed subgoals
    Stored value accepted: $q_i \leftarrow 1$; stored value rejected: $q_i \leftarrow 0$
    If an accepted value differs from K_i, set $q_i \leftarrow 0$
    Malformed actions and unmet predecessors leave q unchanged
    if all subgoals are complete **then**
      break
    end if
  end for
end for

B Protocol, computation, and reproducibility

Budget allocation and missing dependencies. The policy is Qwen3-30B-A3B-Instruct-2507. Its retained matrix contains 816 episodes (24 seeds, two tasks, five selectors at three budgets, plus no/all controls) and 96 missing-edge episodes (12 seeds, two tasks, two selectors, two deletion rates). Primary seeds are 1000–1023; edge conditions use 1000–1011. These are paired environment seeds, not independent samples per API call. The primary focal contrast, fixed before collection, is RD (`risk_dependency`) versus random at four records per update step. Temperature is zero, the output cap is 768 tokens, episode order uses shuffle seed 73, and collection uses the centrally configured bounded transport.

Cross-model comparison. GPT-4.1-mini uses new seeds 2000–2011: 144 episodes with the same four sparse selectors at 20% and both controls. It adds native JSON mode, and a 1500-call cap. GLM-5.3-Flash and Qwen3.6-35B-A3B each use seeds 4000–4023: 288 episodes per model with the same conditions. These two models use native JSON, an 8192-token output cap, a 180-second timeout, and a combined 5800 logical-call cap. GLM uses low reasoning effort; Qwen3.6 disables optional reasoning. Actual request concurrency is eight. Rate-limit rejections alone are logged and retried up to five times, at least 15 seconds apart. An initial rate-limited collection was excluded in full; all conditions were restarted on fresh seeds with these settings. Settings match across selectors within each model; cross-model cost comparisons also reflect different inference settings.

The cross-model conditions were fixed after the Qwen3-30B collection and before their own held-out collection. They are not part of the original pre-specified matrix. Development seeds 0, 1, and 2 are excluded. The final submission contains only the four named models; a removed model condition is excluded from its retained configuration and logs. No retained outcome was altered.

Feedback and joint selection. This experiment uses seeds 5000–5011 and four updates per update step. GPT-4.1-mini contributes 144 episodes: two tasks, 12 seeds, six conditions. Qwen3.6-35B-A3B contributes 96 with four conditions. Both use native JSON, concurrency eight, a 180-second timeout, temperature zero, and no retries. GPT has a 768-token response cap; Qwen has 8192 tokens with optional reasoning disabled. The combined logical-call cap is 2400. The primary follow-up contrast is joint versus feedback-risk, evaluated separately for each model and family. Joint versus static is secondary; GPT also tests feedback-risk–dependency. No condition was removed based on its outcome.

The final sample used 17,937 logical model calls and 25,403,164 reported tokens. There were 3 recorded API failures and 3 calls without usage data; they remain in the evaluation. Recorded list prices imply an estimated $6.50, before any cache discounts and excluding unreported usage. The updater produced correct new predictions for 22,160 of 22,752 selected records (97.4%); missing or malformed updates were not replaced with ground truth.

The excluded interrupted collection logged 1,101 logical requests, including 127 rate-limit failures, and 2,407,902 reported tokens. These are additional diagnostic computation, not part of the final sample or its cost estimate.

The release retains compact episode-level measurements, including model, condition, task seed, selected records, feedback, scores, token counts, and transport-error counts. Deterministic validation checks the episode schema, study counts, identifiers, configured seed ranges, generated job uniqueness, and distinct seeded task worlds. No raw requests, responses, response IDs, historical source snapshots, or separate replay-audit artifacts are released.

The implementation uses the OpenAI SDK with an OpenAI-compatible inference API. Credentials are excluded from the paper-source archive. That archive contains only the LaTeX source and generated tables and vector figures. The repository contains the locked implementation, centralized experiment config, and episode-level evidence needed to rebuild those artifacts without inference. A new run may differ despite temperature zero because provider implementations can change.

C Budget allocation: outcomes across update budgets

Table 4: Complete Qwen3-30B budget sweep on DAG. k: records per update step; n: paired seeds per condition. Progress is mean job completion (%) with descriptive 95% bootstrap intervals; success counts full episodes out of n. Tokens include policy and updater input and output. Blue marks R, D, RD; bold marks the largest budgeted progress mean within each k. Controls are shown once.

k	Selector	n	Progress (%) ↑ [95% CI]	Success ↑	Tokens ↓
0	No updates	24	36.1 [26.9, 45.8]	0/24	13.4k
2	Random	24	39.2 [29.9, 49.0]	0/24	13.8k
2	Even spacing	24	36.6 [27.3, 46.9]	0/24	13.8k
2	R	24	40.8 [30.4, 51.4]	0/24	13.8k
2	D	24	42.0 [34.0, 50.5]	0/24	13.8k
2	RD	24	**46.7** [39.4, 54.3]	0/24	13.6k
4	Random	24	46.0 [36.3, 56.1]	0/24	13.9k
4	Even spacing	24	39.8 [29.9, 49.7]	0/24	14.0k
4	R	24	51.7 [40.8, 62.2]	0/24	13.8k
4	D	24	47.0 [40.8, 53.5]	0/24	13.9k
4	RD	24	**56.9** [50.2, 63.5]	0/24	13.8k
7	Random	24	56.2 [46.9, 65.5]	0/24	14.3k
7	Even spacing	24	43.6 [33.5, 53.8]	0/24	14.4k
7	R	24	60.6 [50.5, 70.0]	1/24	14.0k
7	D	24	56.1 [49.7, 62.7]	0/24	14.3k
7	RD	24	**63.4** [55.4, 70.5]	0/24	14.1k
24	All records	24	98.4 [95.5, 100.0]	22/24	13.2k

Table 5: Complete Qwen3-30B budget sweep on Precedence Chains. Units, intervals, controls, and highlighting follow Table 4.

k	Selector	n	Progress (%) ↑ [95% CI]	Success ↑	Tokens ↓
0	No updates	24	45.0 [36.5, 53.6]	0/24	12.5k
2	Random	24	42.9 [34.2, 52.3]	1/24	12.8k
2	Even spacing	24	44.3 [36.1, 52.8]	0/24	12.9k
2	R	24	47.4 [39.1, 56.2]	1/24	12.8k
2	D	24	46.2 [37.8, 54.9]	0/24	12.9k
2	RD	24	**52.1** [44.3, 59.7]	0/24	12.9k
4	Random	24	54.7 [44.8, 64.8]	1/24	13.0k
4	Even spacing	24	50.0 [41.3, 59.0]	1/24	13.0k
4	R	24	54.9 [44.1, 65.3]	1/24	13.0k
4	D	24	54.9 [48.6, 61.5]	0/24	13.1k
4	RD	24	**62.3** [54.2, 70.5]	1/24	12.8k
7	Random	24	55.2 [45.1, 65.3]	0/24	13.5k
7	Even spacing	24	49.5 [40.3, 58.9]	0/24	13.6k
7	R	24	59.9 [50.2, 70.0]	2/24	13.2k
7	D	24	63.5 [58.5, 68.8]	0/24	13.5k
7	RD	24	**70.1** [62.8, 77.3]	2/24	13.1k
24	All records	24	98.6 [95.8, 100.0]	23/24	12.2k

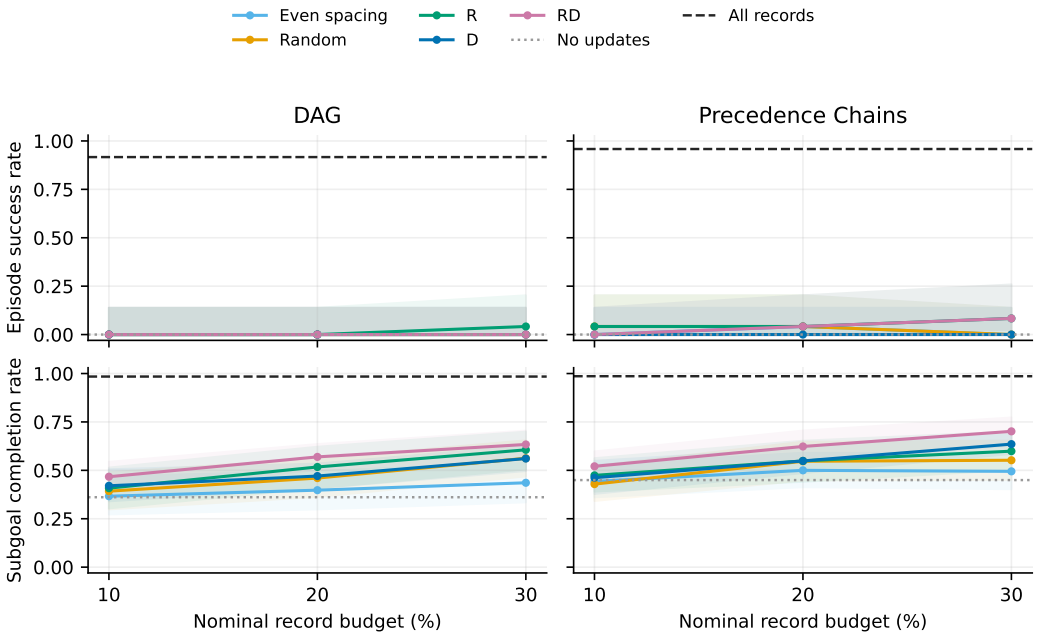


Figure 1: Budget allocation on Qwen3-30B with 24 paired seeds per task. Top: episode success. Bottom: subgoal progress. Shading shows descriptive 95% intervals. Dashed and dotted lines are all-record and no-update controls.

D Missing dependencies: outcomes with incomplete graphs

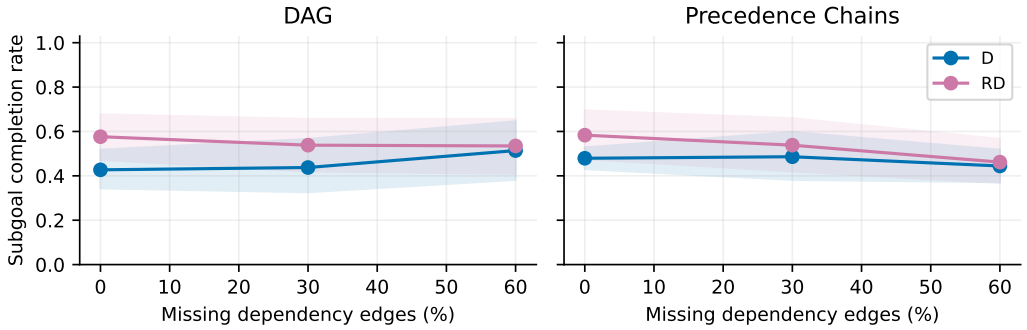


Figure 2: Qwen3-30B at four records per update step, on the same 12 seeds per deletion level. Edges are removed only from the selector's graph; the policy receives identical task specifications. The clean comparison is recomputed on this same 12-seed subset. Shading is a 95% bootstrap interval for progress.

Missing-edge corruption tests incomplete structural information, not a learned extractor. In DAG, the score counts indirect descendants even though actual calls read direct dependencies. In Precedence Chains, an incomplete predecessor additionally blocks later jobs. The same centrality heuristic can therefore misestimate costs for different reasons in the two task structures.

E Cross-model comparison: outcomes by policy

Table 6: Cross-model comparison: GPT-4.1-mini on 12 new seeds per task. Sparse selectors use four records per update step; all-record uses 24. Success counts complete episodes; Progress is a fraction; Tokens include policy and updater; Rejects counts prediction rejections. Blue marks R, D, RD; bold marks the best budgeted outcome in each task.

Task	Selector	Success ↑	Progress ↑	Tokens ↓	Rejects ↓
DAG	Random	0/12	0.63	13.2k	16.9
DAG	R	0/12	0.60	13.2k	17.1
DAG	D	0/12	0.61	13.2k	17.0
DAG	RD	0/12	**0.67**	13.1k	**15.6**
DAG	No updates	0/12	0.51	12.7k	18.9
DAG	All records	12/12	1.00	12.1k	0.0
Precedence Chains	Random	0/12	0.54	12.5k	10.2
Precedence Chains	R	0/12	0.54	12.5k	9.9
Precedence Chains	D	0/12	0.51	12.5k	10.2
Precedence Chains	RD	0/12	**0.57**	12.4k	**9.1**
Precedence Chains	No updates	0/12	0.44	11.9k	10.4
Precedence Chains	All records	12/12	1.00	11.5k	0.0

Table 7: GLM-5.3-Flash, 24 paired seeds per task. Columns and highlighting follow Table 6.

Task	Selector	Success ↑	Progress ↑	Tokens ↓	Rejects ↓
DAG	Random	0/24	0.52	15.2k	13.9
DAG	R	0/24	0.46	14.9k	15.8
DAG	D	0/24	0.59	15.1k	12.8
DAG	RD	**1/24**	**0.60**	14.6k	**12.4**
DAG	No updates	1/24	0.41	14.2k	16.8
DAG	All records	23/24	0.99	13.8k	0.5
Precedence Chains	Random	**1/24**	**0.65**	14.3k	**8.2**
Precedence Chains	R	**1/24**	0.59	14.3k	9.0
Precedence Chains	D	0/24	0.64	14.1k	9.6
Precedence Chains	RD	0/24	0.62	14.5k	9.4
Precedence Chains	No updates	0/24	0.54	13.7k	9.7
Precedence Chains	All records	23/24	0.99	12.8k	0.5

Table 8: Qwen3.6-35B-A3B, 24 paired seeds per task. Columns and highlighting follow Table 6.

Task	Selector	Success ↑	Progress ↑	Tokens ↓	Rejects ↓
DAG	Random	0/24	0.45	15.0k	20.8
DAG	R	0/24	0.38	15.1k	22.3
DAG	D	0/24	0.50	14.9k	19.5
DAG	RD	0/24	**0.52**	14.9k	**18.8**
DAG	No updates	0/24	0.35	14.4k	23.1
DAG	All records	24/24	1.00	13.3k	0.0
Precedence Chains	Random	**1/24**	0.47	13.9k	9.5
Precedence Chains	R	**1/24**	0.44	13.9k	10.1
Precedence Chains	D	0/24	**0.55**	14.0k	10.3
Precedence Chains	RD	0/24	**0.55**	14.0k	**9.4**
Precedence Chains	No updates	0/24	0.39	13.3k	10.3
Precedence Chains	All records	24/24	1.00	12.9k	0.0

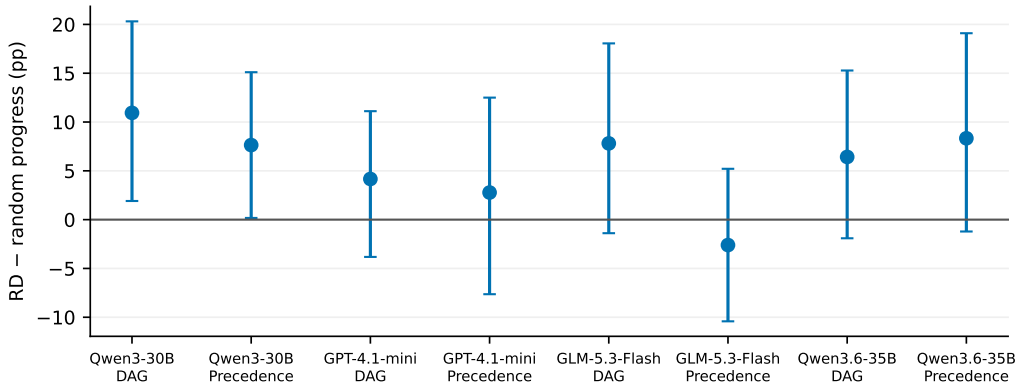


Figure 3: RD minus random subgoal progress at four updates per update step. Error bars resample paired seeds; crossing zero indicates uncertain direction.

Table 9: Cross-model comparison: RD versus random at four records per update step. Progress differences are percentage points with paired 95% intervals (↑); bold if the interval excludes zero. Blue identifies RD as part of our update-selection strategies. Final columns count successes.

Model	Task	Δ progress ↑ (pp, 95% CI)	Random	RD	All
Qwen3-30B	DAG	**+10.9 [+1.9, +20.3]**	0/24	0/24	22/24
Qwen3-30B	Precedence Chains	**+7.6 [+0.2, +15.1]**	1/24	1/24	23/24
GPT-4.1-mini	DAG	+4.2 [-3.8, +11.1]	0/12	0/12	12/12
GPT-4.1-mini	Precedence Chains	+2.8 [-7.6, +12.5]	0/12	0/12	12/12
GLM-5.3-Flash	DAG	+7.8 [-1.4, +18.1]	0/24	1/24	23/24
GLM-5.3-Flash	Precedence Chains	-2.6 [-10.4, +5.2]	1/24	0/24	23/24
Qwen3.6-35B	DAG	+6.4 [-1.9, +15.3]	0/24	0/24	24/24
Qwen3.6-35B	Precedence Chains	+8.3 [-1.2, +19.1]	1/24	0/24	24/24

F Feedback and joint selection: outcomes and costs

Table 10: All feedback and joint selection conditions. Progress is a percentage; Success counts complete episodes out of 12. Tokens include policy and updater. Selection time is mean local computation per update step, excluding inference. ↑ higher is better, ↓ lower. Labels follow Table 1. Blue marks RD, FR, FRD, and Joint; bold marks best progress and success among budgeted selectors.

Task	Selector	Progress ↑	Success ↑	Tokens ↓	Select ms ↓
GPT-4.1-mini					
DAG	Random	63.2	0/12	13.2k	0.08
DAG	RD	66.0	0/12	13.1k	0.21
DAG	FR	69.4	1/12	13.0k	0.19
DAG	FRD	81.6	**2/12**	12.7k	0.18
DAG	Joint	**84.0**	0/12	13.0k	1.73
DAG	All records	100.0	12/12	12.1k	0.08
Precedence Chains	Random	55.2	0/12	12.5k	0.09
Precedence Chains	RD	49.3	0/12	12.5k	0.21
Precedence Chains	FR	66.3	0/12	12.4k	0.22
Precedence Chains	FRD	80.9	0/12	12.3k	0.28
Precedence Chains	Joint	**81.9**	0/12	12.3k	2.18
Precedence Chains	All records	100.0	12/12	11.5k	0.09
Qwen3.6-35B					
DAG	RD	50.3	0/12	14.8k	0.23
DAG	FR	63.9	0/12	14.8k	0.20
DAG	Joint	**80.6**	**1/12**	14.3k	2.57
DAG	All records	100.0	12/12	13.3k	0.08
Precedence Chains	RD	44.1	0/12	14.0k	0.18
Precedence Chains	FR	60.8	0/12	14.0k	0.25
Precedence Chains	Joint	**75.0**	0/12	13.9k	2.49
Precedence Chains	All records	100.0	12/12	12.8k	0.09

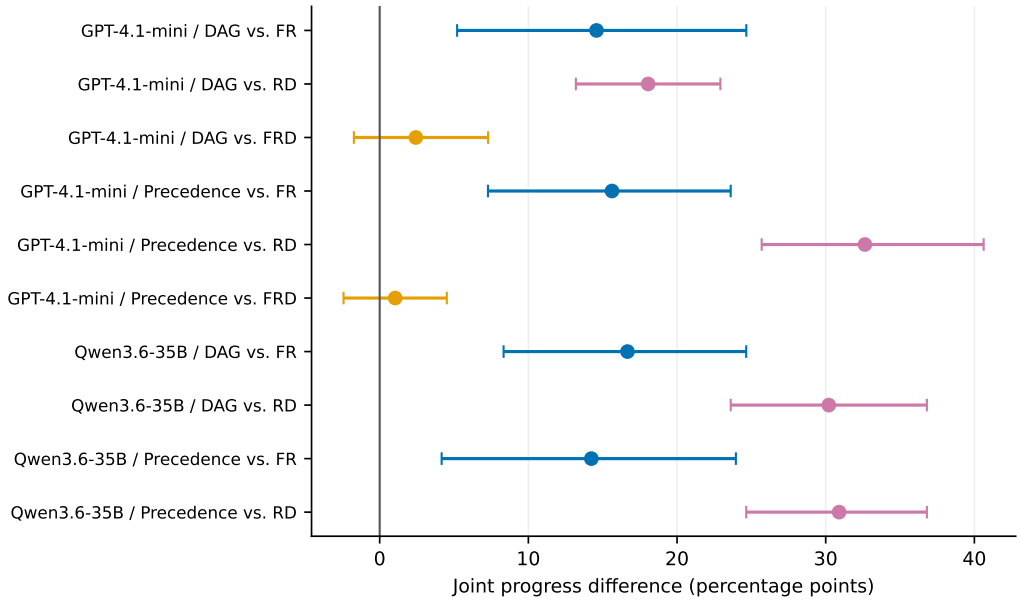


Figure 4: Feedback and joint selection with 12 paired episodes per condition. Points are mean progress differences; bars are 95% paired-bootstrap intervals. Comparators use the same RD, FR, and FRD labels as Table 3. Joint exceeds FR in all four conditions; both GPT comparisons with FRD include zero.

G Updater accuracy and computation

Table 11: Updater outputs by model. Correct counts newly produced predictions, not an old stored prediction that happens to remain correct.

Model	Selected	Parsed	Correct new predictions
Qwen3-30B	9,312	9,288	8,874
GPT-4.1-mini	4,032	4,032	4,032
GLM-5.3-Flash	3,840	3,732	3,732
Qwen3.6-35B	5,568	5,568	5,522

Table 12: Computation by model in the final sample. Token columns are millions; USD is the list-price estimate, before cache discounts.

Model	Episodes	Calls	Input M	Output M	USD
Qwen3-30B	912	8,919	11.48	0.78	1.90
GPT-4.1-mini	288	2,735	3.37	0.25	1.74
GLM-5.3-Flash	288	2,688	3.46	0.65	0.85
Qwen3.6-35B	384	3,595	4.75	0.67	2.02

H Data provenance and assistance

Tasks are generated from seeded configurations, rather than copied from an external benchmark. Repeated conditions intentionally share each task seed; these repetitions are paired in the analysis. Deterministic release validation checks for duplicate initial worlds across distinct task seeds. We do not have access to model pretraining data and make no claim about training-set membership of task concepts. An AI coding assistant supported implementation, experiment orchestration, analysis, and manuscript drafting. Reported measurements come from logged inference and executable validation; package analysis code traces figures and tables to those observations.

I Subgoal Progress and Episode Success

For a single state-change event, independent changes, perfect selected updates, and a policy that must use every record, full success requires all unselected records to remain valid. Thus

$$\Pr(\text{full success} \mid U) = \prod_{i \notin U} (1 - p_i).$$

At equal update cost, this simplified objective selects the largest change probabilities, independently of descendant counts. By contrast, an additive loss for unfinished downstream subgoals can give high value to a shared prerequisite. The measured tasks with two state-change events include ordering constraints, persistent completed subgoals, and imperfect model decisions, so this formula is an explanatory special case, not a prediction substituted for observed results. It shows why an increase in completed subgoals need not imply an increase in complete episodes.

J Record representation and selector implementation

The static family compares R, D, and RD with uniform random sampling without replacement and deterministic even spacing. For n records indexed from zero, even spacing selects $\lfloor (j + 1/2)n/k \rfloor$, $j = 0, \ldots, k-1$; it is not uniform random sampling. Static scores ignore execution feedback. A seeded shuffle breaks score ties. The released identifiers are `change_risk` (R), `dependency_importance` (D), `risk_dependency` (RD), and `even_spacing`. FR and FRD use `feedback_risk` and `feedback_risk_dependency`. FRD uses unfinished requirement counts, not static discounted descendants, and is not a dependency-only selector.

Each state component has a mode and revision. Its predictive record specifies the value `mode@revision` expected to be accepted on its next use. An action supplies these values for its own component and direct dependencies; the verifier compares the submitted values with the hidden state. All records initially store the configuration before environment state changes. Modes have four possible values and initial revisions are integers from one to four. A state change alters mode and increments revision; its random stream is independent of action choices. All records have unit importance weight. Dependency centrality counts each reachable node once at shortest-path distance, and includes the record itself. The risk-only selector uses the supplied change probability, not the realized hidden state change. Graph scoring uses no held-out rewards.

K Validity feedback and the joint objective

A failed action names incorrect submitted state values but does not reveal their correct replacements. A stored value is marked valid only if it was submitted and accepted. If an accepted value differs from the stored prediction, that prediction is marked invalid; rejection of a different submitted value does not certify it. Malformed actions and unmet predecessor constraints provide no validity evidence. At an update step, selected evidence identifies the validated value; a wrong updater output remains wrong in the policy's stored predictions and receives validity zero. Thus the selector can use visible evidence without receiving an unselected hidden state or silently repairing the LLM's mistakes.

For Precedence Chains, each unfinished subgoal requires its own and direct parents' state records. The set also includes requirements of any unfinished predecessor, recursively. A completed predecessor need not be executed again, but its state may still be read by an unfinished child. This distinguishes persistent subgoal completion from prediction validity after a new state change.

For $i \notin U$, the marginal value of updating record i is

$$F(U \cup \{i\}) - F(U) = (1 - q_i) \sum_{j \in \mathcal{P} : i \in A_j} \prod_{\ell \in A_j \setminus (U \cup \{i\})} q_\ell. \tag{3}$$

Every term is nonnegative, so F is monotone. Enlarging U removes factors in $[0, 1]$, so a marginal cannot decrease: F is supermodular. This proves complementarity in the proxy, not optimality for episode return. The formula also covers zero probabilities without division by zero. The empty product equals one. The implementation tests the vectorized exhaustive search against scalar enumeration, including complementary prerequisites and completed-predecessor cases. Candidate scores within 10^{-12} of the maximum are treated as ties; the candidate count is $\binom{n}{k}$.

L Relation to SKL

SKL's update schedules (Sec. 4.1 for uniform subsampling, Secs. 2.1–2.2 for root-only and full bootstrapping, and Sec. 3.2 for SKL-RL simulation) operate on encountered trajectory states (Song et al., 2026). Our random control instead samples component records from a fixed set of stored predictions. Each record predicts a value accepted on next use; SKL learns broader natural-language assessments from aggregated experience. The supplied risks, dependency graph, and explicit validity feedback isolate update allocation here. Applying these selectors to SKL would require defining comparable records and obtaining those signals in its original environments, followed by a separate empirical evaluation.

M Worked Example of Update Selection

This illustrative calculation uses four records, not an additional experiment. Let the supplied graph have edges $a \to c$, $b \to c$, and $c \to d$, and let the knowledge-update budget be $k = 2$. Record a might describe a migration, b a prerequisite, c a service configuration, and d a final check. With the same distance discount as the experiments, $C(a) = C(b) = 2.71$, $C(c) = 1.90$, and $C(d) = 1$.

Table 13: Static example. The three rankings answer different questions.

Record	a	b	c	d
Risk p_i	0.10	0.02	0.13	0.30
Importance $C(i)$	2.71	2.71	1.90	1.00
Product $p_i C(i)$	0.2710	0.0542	0.2470	0.3000

Static priorities. R selects $\{c, d\}$, the two largest change risks. D selects $\{a, b\}$, the two most widely used prerequisites. RD selects $\{a, d\}$: downstream importance moves a ahead of c despite its smaller change risk. None of these rules uses execution feedback.

Feedback-conditioned priorities. Now suppose only subgoals c, d remain unfinished in DAG, with $A_c = \{a, b, c\}$ and $A_d = \{c, d\}$, and current validity estimates $q_a = 0$, $q_b = 0.98$, $q_c = 0$, $q_d = 0.70$. Such estimates combine the initial risks with visible execution feedback. Remaining demand is $D_t(a) = D_t(b) = D_t(d) = 1$ and $D_t(c) = 2$. FR scores are $(1, 0.02, 1, 0.30)$; FRD scores are $(1, 0.02, 2, 0.30)$. Both select $\{a, c\}$, but FRD gives c greater priority because both unfinished subgoals use it. Different scores need not produce different selected sets at a given budget.

Joint selection. The two selected records in $\{a, c\}$ complement each other for subgoal c. The proxy assigns

$$F(\{a, c\}) = q_b + q_d = 1.68, \qquad F(\{c, d\}) = q_a q_b + 1 = 1.$$

The remaining values are $F(\{b, c\}) = 0.70$ and $F(\{a, b\}) = F(\{a, d\}) = F(\{b, d\}) = 0$. Joint therefore also selects $\{a, c\}$. A value of 1.68 means an expected count of subgoals with valid record requirements under the proxy assumptions; it is neither a success probability nor a guarantee that the policy will execute those subgoals correctly.